

Universidad Católica San Pablo

Escuela Profesional de Ciencia de la Computación

Práctica 05: xUnit y Cobertura de Código

Ingeniería de Software II

Alice Gabriela Sucasaca Ilasaca | Prof. DSc. Edgar Sarmiento Calisaya

7 de octubre de 2026

1. Código analizado

Se implementó el método `bubbleSort` del enunciado de la práctica como el método `sort` de la clase `BubbleSorter` (C++, paradigma orientado a objetos), usando `std::vector` (ordenamiento ascendente). El pseudocódigo del enunciado usa índices desde 1 ($i = n..2$, $j = 1..i-1$); en C++ los índices empiezan en 0, por lo que se adaptó a $i = n-1..1$ y $j = 0..i-1$, que es equivalente: en ambos casos cada elemento se compara con su vecino de la derecha. Se convierte `vet.size()` a `int` para que, con un vector vacío, $n - 1$ valga -1 y no un entero sin signo desbordado.

```
src > C BubbleSorter.h
1  #pragma once
2  #include <vector>
3
4  class BubbleSorter
5  {
6  public:
7      void sort(std::vector<int>& vet) const;
8  };
9
```

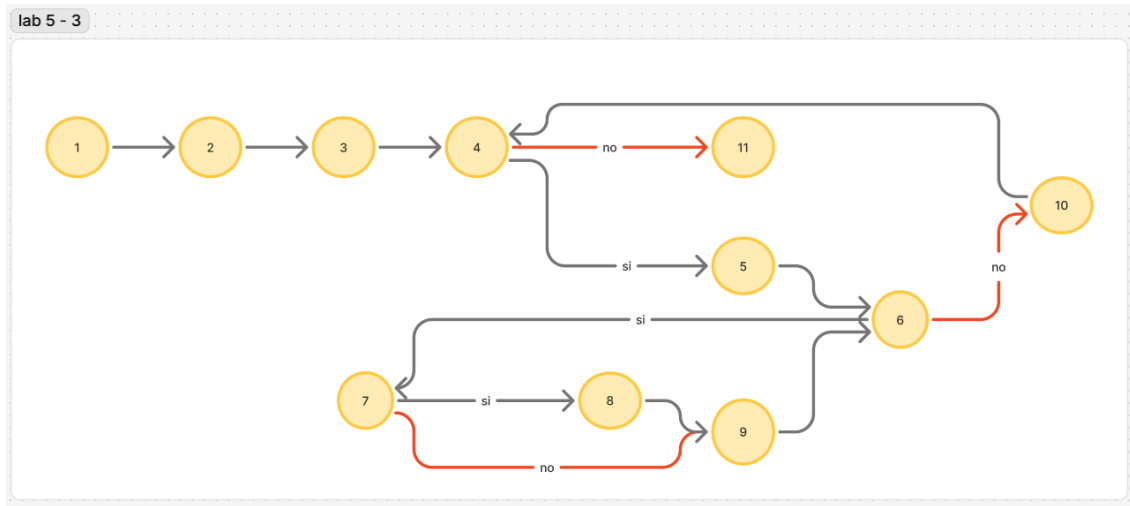
Figura 1: Declaración de la clase (`src/BubbleSorter.h`)

```
src > C BubbleSorter.cpp
1  #include "BubbleSorter.h"
2  #include <utility>
3
4  void BubbleSorter::sort(std::vector<int>& vet) const
5  {
6      int n = static_cast<int>(vet.size()); // 1 Entrada
7      for (int i = n - 1; i >= 1; i--) // 2 n = size
8      { // 3 i = n-1
9          for (int j = 0; j <= i - 1; j++) // 4 i >= 1 ?
10         { // 5 j = 0
11             if (vet[j] > vet[j + 1]) // 6 j <= i-1 ?
12                 std::swap(vet[j], vet[j + 1]); // 7 if v[j] > v[j+1]
13             // 8 swap
14             j++; // 9 j++
15         } //10 i--
16     } //11 Salida
17 }
```

Figura 2: Implementación (`src/BubbleSorter.cpp`) con la numeración de nodos

2. Grafo de Flujo de Control

Se construyó el grafo de flujo de control siguiendo los patrones de las diapositivas de teoría (*Flow graph construction*):

Figura 3: Grafo de Flujo de Control de `BubbleSorter::sort`

Correspondencia nodo – sentencia:

Nodo	Sentencia	Tipo
1	Entrada a la función	Inicio
2	<code>n = vet.size()</code>	Proceso
3	<code>i = n - 1</code>	Proceso
4	<code>i >= 1 ?</code> (bucle externo)	Decisión
5	<code>j = 0</code>	Proceso
6	<code>j <= i - 1 ?</code> (bucle interno)	Decisión
7	<code>if (vet[j] > vet[j+1])</code>	Decisión
8	<code>std::swap(vet[j], vet[j+1])</code> (solo si 7 es verdadera)	Proceso
9	<code>j++</code>	Proceso
10	<code>i--</code>	Proceso
11	Salida de la función	Fin

Cuadro 1: Correspondencia entre nodos del grafo y sentencias del código

Aristas del grafo ($E = 13$): $1 \rightarrow 2$, $2 \rightarrow 3$, $3 \rightarrow 4$, $4 \rightarrow 5$ (sí), $4 \rightarrow 11$ (no), $5 \rightarrow 6$, $6 \rightarrow 7$ (sí), $6 \rightarrow 10$ (no), $7 \rightarrow 8$ (sí), $7 \rightarrow 9$ (no), $8 \rightarrow 9$, $9 \rightarrow 6$, $10 \rightarrow 4$.

3. Complejidad Ciclomática

- Nodos de decisión $P = 3$ (nodos 4, 6 y 7): $V(G) = P + 1 = 4$.
- Aristas $E = 13$ y nodos $N = 11$: $V(G) = E - N + 2 = 13 - 11 + 2 = 4$.
- Regiones del grafo (incluida la exterior): 4.

Por lo tanto, se requieren **4 caminos de ejecución independientes**.

4. Caminos de Ejecución Independientes

Estrategia: **cobertura de decisiones + condición simple** (cada decisión 4, 6 y 7 se evalúa como verdadera y como falsa). Como ninguna condición usa `&&` ni `||`, la condición simple y la múltiple coinciden.

- **Path 1:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 11$ ($n < 2$, no entra a los bucles).
- **Path 2:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 9 \rightarrow 6 \rightarrow \dots \rightarrow 10 \rightarrow 4 \rightarrow \dots \rightarrow 11$ (if = falso, sin swap).
- **Path 3:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9 \rightarrow 6 \rightarrow \dots \rightarrow 10 \rightarrow 4 \rightarrow \dots \rightarrow 11$ (if = verdadero, con swap).
- **Path 4:** múltiples iteraciones de ambos bucles, con el if verdadero y falso. Por ejemplo, con $\{5, 1, 4, 2, 8\}$ la primera pasada externa recorre $\dots \rightarrow 7 \rightarrow 8 \rightarrow 9 \rightarrow 6$ tres veces (swaps), luego $7 \rightarrow 9 \rightarrow 6 \rightarrow 10$ (sin swap), y las pasadas siguientes repiten ambos casos.

Nota: el camino $4 \rightarrow 5 \rightarrow 6 \rightarrow 10$ (bucle interno sin ejecutarse) es infactible: si $i \geq 1$, entonces $j = 0 \leq i - 1$ siempre se cumple.

5. Casos de Prueba

ID CP	Camino en el Grafo	Valores de Prueba	Resultado Esperado
CP1	$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 11$	$\{\}$ y $\{42\}$	$\{\}$ y $\{42\}$
CP2	$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 9 \rightarrow 6 \rightarrow \dots \rightarrow 10 \rightarrow 4 \rightarrow \dots \rightarrow 11$	$\{1, 2, 3, 4\}$	$\{1, 2, 3, 4\}$
CP3	$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 9 \rightarrow 6 \rightarrow \dots \rightarrow 10 \rightarrow 4 \rightarrow \dots \rightarrow 11$	$\{4, 3, 2, 1\}$	$\{1, 2, 3, 4\}$
CP4	Múltiples iteraciones de ambos bucles ($7 \rightarrow 8$ y $7 \rightarrow 9$ en la misma ejecución)	$\{5, 1, 4, 2, 8\}$	$\{1, 2, 4, 5, 8\}$

Cuadro 2: Casos de prueba identificados (cobertura de decisiones + condición simple)

Además se agregó una prueba adicional de frontera con dos elementos: $\{9, 3\} \rightarrow \{3, 9\}$.

6. Implementación de Casos de Prueba (Google Test)

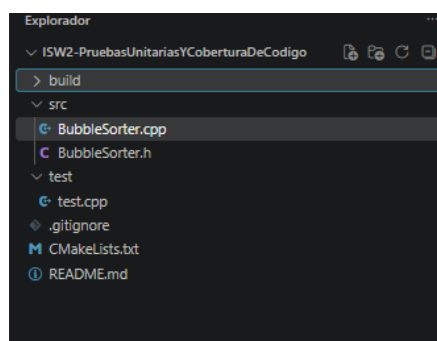


Figura 4: Estructura del proyecto

```

test > test.cpp
1  #include <gtest/gtest.h>
2  #include "BubbleSorter.h"
3  #include <vector>
4
5  class BubbleSortTest : public ::testing::Test
6  {
7  protected:
8      BubbleSorter sorter;
9      std::vector<int> vet;
10
11     void SetUp() override { vet.clear(); }
12     void TearDown() override { vet.clear(); }
13 };
14
15 //1: Outer false (n < 2, no entra a los bucles)
16 TEST_F(BubbleSortTest, EmptyOrSingleElement)
17 {
18     std::vector<int> empty = {};
19     std::vector<int> single = {42};
20     std::vector<int> expectedEmpty = {};
21     std::vector<int> expectedSingle = {42};
22
23     sorter.sort(empty);
24     sorter.sort(single);
25
26     EXPECT_EQ(empty, expectedEmpty);
27     EXPECT_EQ(single, expectedSingle);
28 }
29
30 //2: Outer true, Inner true, if = false (ya ordenado / no swap)
31 TEST_F(BubbleSortTest, AlreadySorted_NoSwap)
32 {
33     vet = {1, 2, 3, 4};
34     std::vector<int> expected = {1, 2, 3, 4};
35     sorter.sort(vet);
36     EXPECT_EQ(vet, expected);
37 }
38
39 //3: Outer true, Inner true, if = true
40 TEST_F(BubbleSortTest, Unsorted_WithSwap)
41 {
42     vet = {4, 3, 2, 1};
43     std::vector<int> expected = {1, 2, 3, 4};
44     sorter.sort(vet);
45     EXPECT_EQ(vet, expected);
46 }
47
48 // 4: mezcla de swaps y noswap
49 TEST_F(BubbleSortTest, MixedCase)
50 {
51     vet = {5, 1, 4, 2, 8};
52     std::vector<int> expected = {1, 2, 4, 5, 8};
53     sorter.sort(vet);
54     EXPECT_EQ(vet, expected);
55 }
56
57 // prueba1 dos elementos desordenados
58 TEST_F(BubbleSortTest, TwoElementsSwap)
59 {
60     vet = {9, 3};
61     std::vector<int> expected = {3, 9};
62     sorter.sort(vet);
63     EXPECT_EQ(vet, expected);
64 }
65
66 int main(int argc, char **argv)
67 {
68     ::testing::InitGoogleTest(&argc, argv);
69     return RUN_ALL_TESTS();
70 }

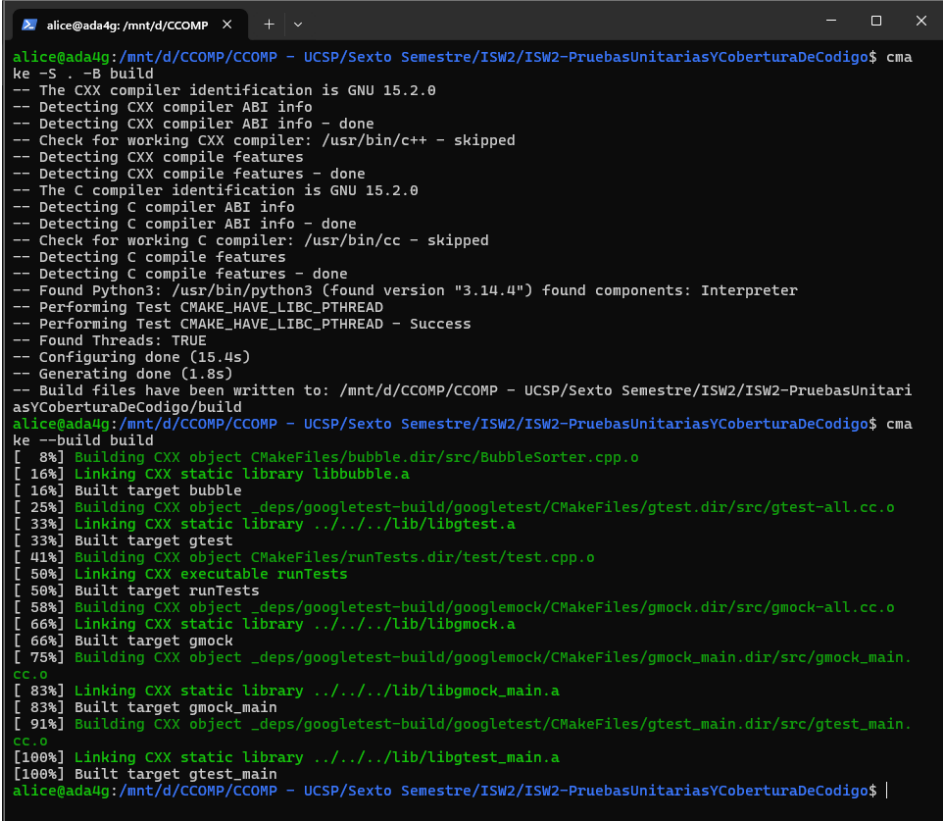
```

Figura 5: Código de test.cpp: fixture con SetUp/TearDown y pruebas CP1–CP4 y adicional

ID CP	Prueba en GTest	Criterio	Resultado
CP1	EmptyOrSingleElement	Outer = falso	PASSED
CP2	AlreadySorted_NoSwap	If = falso (sin swap)	PASSED
CP3	Unsorted_WithSwap	If = verdadero (swap)	PASSED
CP4	MixedCase	Ambas ramas + bucles	PASSED
Extra	TwoElementsSwap	Frontera $n = 2$	PASSED

Cuadro 3: Trazabilidad entre casos de prueba (CP) y pruebas automatizadas

7. Reporte de Ejecución de Pruebas



```

alice@ada4g: /mnt/d/CCOMP x + v
alice@ada4g: /mnt/d/CCOMP/CCOMP - UCSP/Sexto Semestre/ISW2/ISW2-PruebasUnitariasYCoberturaDeCodigo$ cmake
-- The CXX compiler identification is GNU 15.2.0
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working CXX compiler: /usr/bin/c++ - skipped
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- The C compiler identification is GNU 15.2.0
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Check for working C compiler: /usr/bin/cc - skipped
-- Detecting C compile features
-- Detecting C compile features - done
-- Found Python3: /usr/bin/python3 (found version "3.14.4") found components: Interpreter
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD
-- Performing Test CMAKE_HAVE_LIBC_PTHREAD - Success
-- Found Threads: TRUE
-- Configuring done (15.4s)
-- Generating done (1.8s)
-- Build files have been written to: /mnt/d/CCOMP/CCOMP - UCSP/Sexto Semestre/ISW2/ISW2-PruebasUnitari
asYCoberturaDeCodigo/build
alice@ada4g: /mnt/d/CCOMP/CCOMP - UCSP/Sexto Semestre/ISW2/ISW2-PruebasUnitariasYCoberturaDeCodigo$ cmake
--build build
[ 8%] Building CXX object CMakeFiles/bubble.dir/src/BubbleSorter.cpp.o
[ 16%] Linking CXX static library libbubble.a
[ 16%] Built target bubble
[ 25%] Building CXX object _deps/googletest-build/googletest/CMakeFiles/gtest.dir/src/gtest-all.cc.o
[ 33%] Linking CXX static library ../../lib/libgtest.a
[ 33%] Built target gtest
[ 41%] Building CXX object CMakeFiles/runTests.dir/test/test.cpp.o
[ 50%] Linking CXX executable runTests
[ 50%] Built target runTests
[ 58%] Building CXX object _deps/googletest-build/googletest/CMakeFiles/gmock.dir/src/gmock-all.cc.o
[ 66%] Linking CXX static library ../../lib/libgmock.a
[ 66%] Built target gmock
[ 75%] Building CXX object _deps/googletest-build/googletest/CMakeFiles/gmock_main.dir/src/gmock_main.
cc.o
[ 83%] Linking CXX static library ../../lib/libgmock_main.a
[ 83%] Built target gmock_main
[ 91%] Building CXX object _deps/googletest-build/googletest/CMakeFiles/gtest_main.dir/src/gtest_main.
cc.o
[100%] Linking CXX static library ../../lib/libgtest_main.a
[100%] Built target gtest_main
alice@ada4g: /mnt/d/CCOMP/CCOMP - UCSP/Sexto Semestre/ISW2/ISW2-PruebasUnitariasYCoberturaDeCodigo$ |

```

Figura 6: Configuración y compilación con CMake en Linux (GCC)

```

alice@ada4g:/mnt/d/CCOMP/CCOMP - UCSP/Sexto Semestre/ISW2/ISW2-PruebasUnitariasYCoberturaDeCodigo$ ./build/runTests
[=====] Running 5 tests from 1 test suite.
[=====] Global test environment set-up.
[=====] 5 tests from BubbleSortTest
[ RUN      ] BubbleSortTest.EmptyOrSingleElement
[ OK       ] BubbleSortTest.EmptyOrSingleElement (0 ms)
[ RUN      ] BubbleSortTest.AlreadySorted_NoSwap
[ OK       ] BubbleSortTest.AlreadySorted_NoSwap (0 ms)
[ RUN      ] BubbleSortTest.Unsorted_WithSwap
[ OK       ] BubbleSortTest.Unsorted_WithSwap (0 ms)
[ RUN      ] BubbleSortTest.MixedCase
[ OK       ] BubbleSortTest.MixedCase (0 ms)
[ RUN      ] BubbleSortTest.TwoElementsSwap
[ OK       ] BubbleSortTest.TwoElementsSwap (0 ms)
[=====] 5 tests from BubbleSortTest (0 ms total)

[=====] Global test environment tear-down
[=====] 5 tests from 1 test suite ran. (0 ms total)
[ PASSED  ] 5 tests.

```

Figura 7: Ejecución de las pruebas unitarias con Google Test

Resultado: todas las pruebas pasaron (PASSED – 5/5).

8. Reporte de Cobertura de Código

Se utilizó **gcov** como herramienta de cobertura de código junto con el compilador GCC. La herramienta se habilita desde el sistema de construcción, con las opciones `--coverage -O0 -g` en la biblioteca y en el ejecutable de pruebas (fragmento de `CMakeLists.txt`):

```

if(NOT MSVC)
    foreach(t bubble runTests)
        target_compile_options(${t} PRIVATE --coverage -O0 -g)
        target_link_options(${t} PRIVATE --coverage)
    endforeach()
endif()

```

Figura 8: (fragmento de `CMakeLists.txt`)

Después de ejecutar las pruebas se generó el reporte de `BubbleSorter.cpp`, y se verificó que todas sus líneas y ramas fueron alcanzadas.

```

alice@ada4g:/mnt/d/CCOMP/CCOMP - UCSP/Sexto Semestre/ISW2/ISW2-PruebasUnitariasYCoberturaDeCodigo/build/CMakeFiles/bubble.dir$ cd src
alice@ada4g:/mnt/d/CCOMP/CCOMP - UCSP/Sexto Semestre/ISW2/ISW2-PruebasUnitariasYCoberturaDeCodigo/build/CMakeFiles/bubble.dir/src$ gcov -b -c BubbleSorter.cpp.gcov | grep -A5 "BubbleSorter.cpp"
File '/mnt/d/CCOMP/CCOMP - UCSP/Sexto Semestre/ISW2/ISW2-PruebasUnitariasYCoberturaDeCodigo/src/BubbleSorter.cpp'
Lines executed:100.00% of 7
Branches executed:100.00% of 6
Taken at least once:100.00% of 6
Calls executed:100.00% of 6
Creating 'BubbleSorter.cpp.gcov'
alice@ada4g:/mnt/d/CCOMP/CCOMP - UCSP/Sexto Semestre/ISW2/ISW2-PruebasUnitariasYCoberturaDeCodigo/build/CMakeFiles/bubble.dir/src$ |

```

Figura 9: Resumen de cobertura de `BubbleSorter.cpp` generado con `gcov`

```

alice@adaug:/mnt/d/CCOMP/CCOMP - UCSP/Sexto Semestre/ISW2/ISW2-PruebasUnitariasYCoberturaDeCodigo/build/CMakeFiles/bubble.dir/src$ cat BubbleSorter.cpp.gcov
-: 0:Source:/mnt/d/CCOMP/CCOMP - UCSP/Sexto Semestre/ISW2/ISW2-PruebasUnitariasYCoberturaDeCodigo/src/BubbleSorter.cpp
-: 0:Graph:BubbleSorter.cpp.gcov
-: 0:Data:BubbleSorter.cpp.gcov
-: 0:Runs:2
-: 1:#include "BubbleSorter.h"
-: 2:#include <utility>
-: 3:
function_ZNM12BubbleSorter::sort(std::vector<int>& vet) const called 6 returned 100% blocks executed 100%
-: 4:void BubbleSorter::sort(std::vector<int>& vet) const
-: 5:{
-: 6:     int n = static_cast<int>(vet.size()); // 1 Entrada // 2 n = size
call_0 returned 6
-: 7:     for (int i = n - 1; i >= 1; i--) // 3 i = n-1
branch_0 taken 11
branch_1 taken 6 (fallthrough)
-: 8:     { // 4 i >= 1 ?
-: 9:         for (int j = 0; j <= i - 1; j++) // 5 j = 0
branch_0 taken 23
branch_1 taken 11 (fallthrough)
-: 10:         { // 6 j <= i-1 ?
-: 11:             if (vet[j] > vet[j + 1]) // 7 if v[j] > v[j+1]
call_0 returned 23
call_1 returned 23
branch_2 taken 11 (fallthrough)
branch_3 taken 12
-: 12:             std::swap(vet[j], vet[j + 1]); // 8 swap
call_0 returned 11
call_1 returned 11
call_2 returned 11
-: 13:         } // 9 j++
-: 14:     } //10 i--
-: 15: } //11 Salida
alice@adaug:/mnt/d/CCOMP/CCOMP - UCSP/Sexto Semestre/ISW2/ISW2-PruebasUnitariasYCoberturaDeCodigo/build/CMakeFiles/bubble.dir/src$

```

Figura 10: Análisis línea por línea y ejecución de ramas en BubbleSorter.cpp.gcov

Métrica (BubbleSorter.cpp)	Resultado
Líneas ejecutadas	100 % (7 de 7)
Ramas ejecutadas	100 % (6 de 6)
Ramas tomadas al menos una vez	100 % (6 de 6)
Llamadas ejecutadas	100 % (6 de 6)

Cuadro 4: Cobertura de código

Resultado: los 5 casos de prueba cubren el 100 % de las líneas y de las ramas de `BubbleSorter::sort` (decisiones 4, 6 y 7 en verdadero y falso). Los conteos de gcov coinciden con lo esperado: el bucle externo es verdadero 11 veces y falso 6, el interno es verdadero 23 y falso 11, y se hacen 11 swaps.

9. Repositorio en GitHub

El proyecto completo (código, pruebas, CMake e informe) está disponible en: <https://github.com/ekials/ISW2-PruebasUnitariasYCoberturaDeCodigo>

10. Conclusiones

- Se construyó el grafo de flujo de control de `BubbleSorter::sort` (11 nodos y 13 aristas) siguiendo los patrones de las diapositivas de teoría.
- La complejidad ciclomática es $V(G) = 4$, lo que determina 4 caminos independientes y 4 casos de prueba (CP1–CP4).
- Se diseñaron los casos de prueba con la estrategia de cobertura de decisiones + condición simple.
- Se automatizaron las pruebas con Google Test (xUnit), usando `SetUp`, `TearDown` y la estructura Arrange – Act – Assert (inicializar, ejecutar y verificar). Resultado: 5/5 PASSED.
- Con gcov se obtuvo 100 % de cobertura de líneas y de ramas.